

HieraSparse: Hierarchical Semi-Structured Sparse KV Attention

Haoxuan Wang

College of Computing and Data Science
Nanyang Technological University
Singapore, Singapore
haoxuan.wang@ntu.edu.sg

Chen Wang

College of Computing and Data Science
Nanyang Technological University
Singapore, Singapore
chen.wang@ntu.edu.sg

Abstract—The deployment of long-context Large Language Models (LLMs) poses significant challenges due to the intense computational cost of self-attention and the substantial memory overhead of the Key-Value Cache (KV Cache). In this paper, we introduce *HieraSparse*, a hierarchical KV Cache compression framework with acceleration kernels that leverage GPU sparse tensor cores to speed up semi-structured KV Cache attention for both the prefill and decode phases. With the hierarchical design, our method allows for a flexible quality-sparsity trade-off and successfully converts sparsity into efficiency. Compared to the state-of-the-art decode method that utilizes unstructured sparsity, *HieraSparse* achieves $1.2\times$ KV compression ratio and $4.57\times$ attention speedup at the same sparsity level. Furthermore, we extended the semi-structured KV Cache pruning to the prefill stage, which demonstrated up to $1.85\times$ attention speedup at the highest sparsity. Lastly, we evaluate the generation quality of *HieraSparse* with a simple magnitude-based pruning method, and the results show that $1.34\times$ prefill and $1.71\times$ decode attention speedup can be achieved without significant quality drop. The codebase can be found at <https://github.com/psl-ntu/HieraSparse>.

Index Terms—Sparse GEMM, Sparse Attention, KV Cache, Kernel Optimization, Semi-structured Sparsity

I. INTRODUCTION

Large language models are constantly evolving to support longer context windows, showing dominating capabilities in long-context natural language processing [1]–[4] and downstream tasks like code generation [5], text summarization [6], and logical reasoning [7]. Concurrently, application-level demand for long-context processing continues to grow. Emerging paradigms such as Retrieval-Augmented Generation (RAG) [8], In-Context Learning [9], and memory-augmented agentic systems [10] require models to process vast amounts of retrieved documents, extensive demonstrations, or long-term interaction histories. These demands are pushing context window requirements to hundreds of thousands or even millions of tokens [11]–[13], imposing significant pressure on both computation and memory.

Computation. This massive increase in sequence length exposes the quadratic complexity $O(n^2)$ of the self-attention mechanism [14] as a critical bottleneck. As illustrated in Figure 1, attention computation can take 50% of prefill latency at the context length of 64K and continue growing to 80% at 192K, accounting for even minutes of time-to-first-token

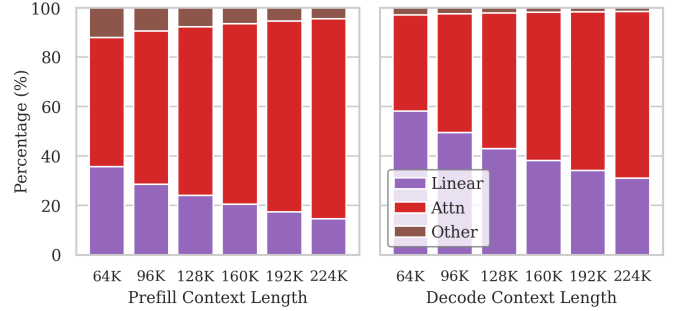


Fig. 1: The latency breakdown of the prefill and decode phases under different context lengths. The attention mechanism gradually dominates the computation as the context length increases.

(TTFT) latency. The decode phase is similarly affected: as the model must attend over all preceding tokens and their associated KV Cache entries, attention can consume more than 60% of time-per-output-token (TPOT) latency under long-context settings. These observations motivate the development of efficient attention mechanisms capable of scaling to long contexts.

Memory. Beyond computation, the KV Cache introduces a memory footprint that scales linearly with sequence length. As a concrete example, serving *Llama-3.1-8B-Instruct* [2] at its maximum supported context length requires 16 GiB of KV Cache memory per request, already on par with the model weights themselves. With context extension techniques [11]–[13], context lengths can readily scale to over one million tokens (1048K), demanding upwards of 125 GiB of KV Cache memory per request, far exceeding the capacity of most modern GPUs. This underscores the pressing need for methods that reduce the memory footprint of KV Cache during LLM inference.

Prior work [15]–[22], [24]–[27] has shown that the KV Cache includes many low-importance entries that can be pruned without significantly degrading generation quality. As summarized in Table I, such pruning techniques operate at varying levels of granularity. Coarse-grained methods prune at the level of tokens, channels, heads, or layers, achieving

TABLE I: KV Cache pruning scheme comparison.

Granularity	Target	Example Method	Flexibility	Algorithm Agnostic	Speedup Mechanism	Speedup	Supported Phase	
							Prefill	Decode
Coarse-grained	Token	<i>H2O</i> [15], <i>SnapKV</i> [16]	Low	No	Computation Skip	High	×	✓
	Channel	<i>ThinK</i> [17], <i>LeanK</i> [18]					×	✓
	Head	<i>HeadKV</i> [19], <i>AdaKV</i> [20]					×	✓
	Layer	<i>PyramidKV</i> [21], <i>DynamicKV</i> [22]					×	✓
	Head+Token	<i>DuoAttention</i> [23]					✓	✓
Fine-grained unstructured	Element	<i>MUSTAFAR</i> [24]	High	Yes	Load-As-Sparse Compute-As-Dense	Low	×	✓
Hierarchical semi-structured	Block+Element	<i>HieraSparse</i> (Ours)	High	Yes	Sparse Tensor Core	High	✓	✓

substantial speedups by skipping computation entirely, though at the cost of reduced flexibility and coarse control over the sparsity-quality tradeoff. Fine-grained, unstructured element-wise pruning, by contrast, offers precise control over which matrix entries are removed, but it fails to translate sparsity into practical efficiency gains. This is because existing sparse execution frameworks [28]–[30] adopt a *load-as-sparse, compute-as-dense* scheme: while memory bandwidth is reduced, the compute cost remains unchanged, rendering such approaches ineffective for compute-bound operations such as large-scale GEMM and prefill attention [31], [32]. Furthermore, the irregular structure inherent in unstructured pruning introduces non-trivial compression overhead: approximately 12% additional latency during prefill in our experiments. Overall, these constraints create a fundamental gap between achieved sparsity and realized efficiency in existing systems.

N:M semi-structured sparsity has emerged as a promising middle ground, where exactly N out of every M consecutive elements are non-zero. Modern GPU architectures (e.g., NVIDIA Ampere, Hopper, and AMD MI300X) provide dedicated sparse tensor cores that can accelerate N:M sparse computations, effectively doubling throughput [33]–[35]. Given the limitations of unstructured-sparsity systems outlined above, applying semi-structured sparsity to the KV Cache is a natural and compelling direction. However, doing so in practice introduces several non-trivial system challenges.

First, to preserve generation quality, sparsity must be applied selectively: critical attention regions—such as attention sinks, heavy hitters, and local windows [15], [25], [36]—must remain dense to retain complete information. This necessitates a hierarchical sparsity design capable of efficiently mixing dense and sparse regions. Second, since sparse tensor cores only compress the first matrix operand, realizing hardware acceleration across both the prefill and decode phases requires a redesign of the attention computation. Third, performing online KV Cache compression risks a *compression latency tax*, where format conversion overhead negates the gains of sparse kernels. Avoiding this requires a highly efficient compressor that can integrate seamlessly into the prefill phase without inflating TTFT.

To address these challenges, we present *HieraSparse*, a framework that enables hardware-accelerated semi-structured sparsity for KV Cache compression. To the best of our knowledge, *HieraSparse* is the first system to make the Key

and Value caches the sparse operands of the GPU sparse tensor core, and the first to extend semi-structured sparse KV Cache acceleration to both the prefill and decode phases of autoregressive LLMs. Our key contributions are:

- We propose a hierarchical block-based memory management scheme that supports mixing of dense and sparse KV Cache blocks, which allows a flexible quality-efficiency trade-off.
- We design highly optimized sparse attention kernels that integrate sparse tensor cores, accelerating both prefill and decode phases with N:M structured sparsity. We also performed a detailed analysis of theoretical and actual speedup under different sparsity settings.
- We implement highly efficient compression kernels that enable near-zero-overhead online KV Cache sparsification.

II. BACKGROUND AND RELATED WORKS

Attention The core component of the transformer is the scaled dot-product attention mechanism [14]. Given three matrices: Query $Q \in \mathbb{R}^{n \times d}$, Key $K \in \mathbb{R}^{n \times d}$, and Value $V \in \mathbb{R}^{n \times d}$, where n is the number of tokens and d is the hidden dimension, the output $O \in \mathbb{R}^{n \times d}$ is computed as:

$$O = \text{softmax} \left(\overbrace{\left(\overbrace{\frac{Q \times K^T}{\sqrt{d}}}^S \right)}^P \right) \times V \quad (1)$$

where $\times$ denotes a general matrix multiplication (GEMM) operation, P and S denote the intermediate results.

Naive attention implementations require $O(n^2)$ space complexity to materialize the intermediate attention score matrix P . To reduce this memory footprint, *FlashAttention* [37], [38] fuses two GEMMs and softmax operations into a single kernel by tiling the computation. This approach maintains row-wise statistics L_i (log-sum-exp) and M_i (max) for the online safe-softmax algorithm, avoiding the need to store the full attention matrix. Additionally, the underlying GEMM operations leverage dedicated hardware acceleration, such as tensor core or matrix core, for maximum throughput. *PagedAttention* [39] as a memory-efficient attention variant designed for model inference, solved memory fragmentation by partitioning the

key and value tensors into smaller blocks that can be loaded and processed individually.

KV Cache Optimization There have been various works that optimize the KV Cache from different perspectives. For example, KV Cache is often offloaded to CPU memory or NVMe device for future reuse when multiple requests share the same prefix (i.e., multi-turn conversation, common document/knowledge question-answering (QA), in-context learning), which has been widely adopted in popular inference engines like *vLLM* and *SGLang* [39], [40]. Frontier works like *CacheBlend* [41], *PromptCache* [42], and *CacheLink* [43] also try to reuse KV Cache at a finer-grained scope, especially when the sharing-prefix condition can not be met. Another batch of works, including different methods of KV Cache quantization and pruning [44], [45], tried to optimize the KV Cache by utilizing its internal numeric characteristics. As shown in Figure 2, some channels in keys consistently exhibit large magnitude across all tokens, while value cache tends to have more uniform small magnitude without a distinguishable pattern. *ThinK* [17] leveraged this observation to remove trivial channels in both query and value matrices, reducing the overall computation and memory bandwidth. *MUSTAFAR* [24] further explored unstructured KV Cache pruning with a magnitude-based algorithm and performed various experiments to analyze the effect of different pruning dimensions, showing that key cache is suitable for per-token pruning due to its outlier channels, while per-token and per-channel pruning show minimal difference for value cache. *ZipKV* [46] proposed a new area where compressed KV Cache can be shared among different prompts that have the same prefixes, which can be done offline and introduces zero runtime overhead.

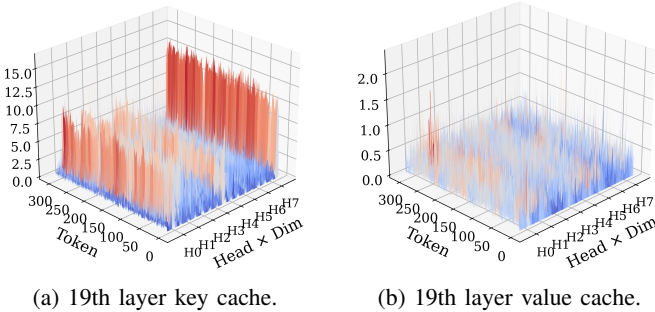


Fig. 2: Visualizations of the key and value cache absolute value from *Llama-3.1-8B-Instruct* using text sliced from *LongBench*. The key cache shows outlier ridges in certain channels and is consistent across all tokens, while the value cache appears more random without distinguishable patterns.

Sparse Tensor Core Since the introduction of the NVIDIA Ampere architecture (and AMD MI300X matrix cores), sparse tensor cores have been designed to accelerate semi-structured GEMM operations of the form $D = A \times B + C$, where matrix A exhibits N:M structured sparsity along the reduction dimension. For example, alongside the dense tensor core instruction `mma.m16n8k16`, there exists a corresponding

sparse instruction, `mma.sp.m16n8k32`, which operates on 2:4 sparse matrices. The sparse instruction processes only the nonzero elements of matrix A , together with additional metadata E that encodes their positions. Importantly, both dense and sparse instructions ideally require the same number of hardware cycles. As a result, sparse tensor cores can achieve up to $2\times$ computational throughput while reducing memory traffic for matrix A to approximately 75% of the dense case. *DFSS* [47] first applied N:M sparse tensor cores to attention by dynamically pruning the attention-score matrix P to 2:4 sparsity for the $P \times V$ product in encoder-style (e.g., BERT) models. This requires materializing the full attention-score matrix, and since P is a dynamic activation it must be pruned online at every step. Moreover, this scheme neither compresses the KV Cache nor applies to the autoregressive decode phase, where no full score matrix is materialized.

III. METHOD

Figure 3 demonstrates the overall workflow of *HieraSparse*. Given the KV Cache that is divided into sparse and dense regions, the caches are further split into blocks. For dense blocks, they are directly stored in the dense cache memory pool; for sparse blocks, they are further pruned and compressed into non-zero data and metadata, then stored in the respective memory pools. A block index mapping is created accordingly, representing the block-level sparsity pattern of KV Cache and tracking the offset of respective blocks in the memory pool. During attention computation, the block index mapping and the memory pools are fed to attention kernels, which utilize the sparse tensor core for acceleration. After the prefill phase, the dense cache can be further pruned and compressed, which allows different sparsity settings for the prefill and decode phases.

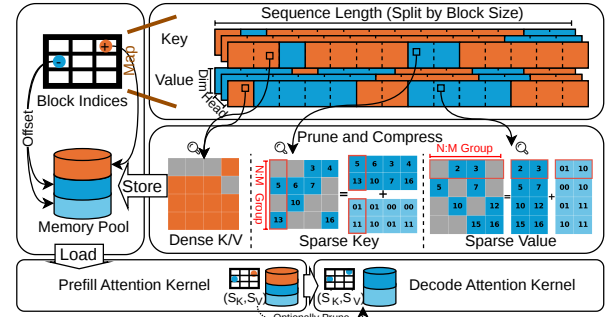


Fig. 3: The overall workflow of *HieraSparse*, which supports flexibility at different levels: i) Different sparsity patterns for the prefill and decode phases. ii) Different sparsity patterns for the key and value caches. iii) Different block-level sparsity pattern. iv) Different element-level sparsity pattern.

There are three main function components in *HieraSparse*: i) Hierarchical cache pruner. ii) Bundled compression kernels that take pruned tensors and output respective non-zero values and metadata. iii) GPU acceleration kernels for both prefill

and decode phases. Each of them will be introduced in the following subsections.

A. Hierarchical Cache Pruner

The cache pruner takes a pruning algorithm that produces multi-level masks: a block-level pruning mask M and an element-level pruning mask m to generate the compressed caches. The block-level mask M determines which blocks are kept dense or pruned to sparse, while the element-level mask m determines which elements within the sparse blocks are pruned to zero.

HieraSparse is designed to be agnostic to the pruning algorithms. For example, the system can be configured with masks produced by *ThinK* or *LeanK* for key cache channel pruning when the mask satisfies or can be permuted to a channel-wise N:M pattern. For simplicity and comparability with unstructured methods, we adopt a straightforward magnitude-based pruning method to generate the block and element-level masks by default. We first divide the KV Cache into blocks along the sequence dimension by a size of B , each block is denoted by K_i and V_i . Within each block, we select every N out of M elements with the lowest magnitude, forming the element-level mask m . For each block, we compute the magnitude loss sum L_i incurred by applying the element-level mask and sort the blocks based on L_i . Finally, we select a portion S of blocks with the lowest L_i to prune to sparse, while the rest are kept dense. Formally, the pruning process can be described as Equations 2:

$$\mathcal{T}_i = \text{top}_{N:M}(|X_i|) \quad (2a)$$

$$m_{X_i} = \mathbb{I}(|X_i| \geq \mathcal{T}_i) \quad (2b)$$

$$L_{X_i} = \|X_i \odot (1 - m_{X_i})\|_1 \quad (2c)$$

$$M_X = \mathbb{I}(L_{X_i} \geq \mathcal{T}_{S_X}), \quad \mathcal{T}_S = \text{top}_{S_X}(\{L_X\}) \quad (2d)$$

Where $X \in \{K, V\}$, $\mathbb{I}(\cdot)$ is the indicator function, $\text{top}_{N:M}(\cdot)$ returns the threshold value to keep the top N elements out of every M elements. Finally, $\text{top}_S(\cdot)$ determines the loss threshold to maintain the target block sparsity S_X .

B. Cache Compressor

Given the block-level mask M and element-level mask m produced by the hierarchical cache pruner, the cache compressor transforms the original dense KV cache into mixed KV Cache blocks suitable for the acceleration kernels. Following memory management strategies from *PageAttention*, we organize dense data, nonzero data, and metadata into separate memory pools, and maintain a block-level index map to track them: we first allocate memory pools for dense blocks, nonzero data blocks, and metadata blocks, along with a block index map. Two counters are initialized to track the number of dense and sparse blocks, respectively. For each block in the original KV Cache, we consult the block-level mask M . If a block is marked as dense, it is directly copied into the dense memory pool, and the dense counter is incremented. Otherwise, the compressor applies the element-level mask m to extract nonzero elements and their corresponding metadata,

which are stored in the sparse data and metadata pools, respectively, and the sparse counter is incremented.

The block index map encodes both the type and location of each block using a sign convention: a positive index denotes a dense block and points to its offset in the dense pool, while a negative index denotes a sparse block and points to its offset in the sparse pool. During attention computation, kernels can efficiently dispatch to the appropriate representation by consulting this index map. It is worth noting that compression itself incurs a latency cost that may partially offset the throughput gains from the acceleration kernels. We address this in Section IV-B, where we describe an efficient implementation that reduces compression overhead to near zero.

C. GPU Acceleration Kernels

1) Fully Semi-Structured Sparse Computation Workflow:

We first describe the design that computes on sparse cache blocks, and later extend it to support mixed dense and sparse blocks. Since the N:M sparse tensor core requires the first input matrix to be semi-structured sparse along the reduction dimension, careful kernel design is required to determine which matrices should be compressed. Table II compares potential strategies based on transposing the two consequent GEMM operations to switch the sparse operands, highlighting their algorithmic support and ideal speedup.

TABLE II: Design space exploration to apply sparse tensor core in attention. $(\cdot)$ is to denote the $\cdot$ has to be same as the shape of GEMM1 output.

Config	GEMM1	GEMM2	Sparse Ops	Algo. Support	Prefill	Decode
Naive	$S = Q \times K^T$	$O = P \times V$	Q, P	Online-Only	2×	1.0×
Trans-K	$S^T = K \times Q^T$	$O = (P^T)^T \times V$	K, P	Mixed	2×	1.5×
Trans-V	$S = Q \times K^T$	$O^T = V^T \times (P)^T$	Q, V	Mixed	2×	1.5×
Trans-Both	$S^T = K \times Q^T$	$O^T = V^T \times P^T$	K, V	Online/Offline	2×	2×

We analyze the four combinations of the GEMM orientation:

- **Naive:** Allows Q and P to be sparse. Since both Q and attention scores P are dynamic activations generated at runtime, they cannot be pre-compressed. While the sparse GEMM itself is 2×

From the analysis above, we choose the **Trans-Both** configuration. The attention computation can be reformulated as

Algorithm 1, where X_{nnz} and X_e represent non-zero and metadata counterparts of a dense cache X . Although the **Trans-Both** scheme appears mathematically straightforward, implementing it effectively on GPUs is nontrivial because the operands of tensor cores are often stored in private registers per thread, which requires a re-layout between two GEMMs. In Section IV-C2, we will describe the detailed implementation of *RelayoutFragment* without using expensive communication like shared memory or warp shuffling.

Algorithm 1 The algorithm for fully sparse attention

Require: $Q, K_{nnz}, K_e, V_{nnz}, V_e$, block sizes B_r, B_c
 Let $T_r = \lceil n/B_r \rceil$ and $T_c = \lceil n/B_c \rceil$, divide $Q, K_{nnz}, K_e, V_{nnz}, V_e$ into corresponding blocks.
for $i = 1$ **to** T_r **in parallel do**
 Initialize O_i^T, L_i, M_i
 $Q_i^T \leftarrow \text{LoadDense}(Q, i)$
 for $j = 1$ **to** T_c **do**
 $K_{nnz_j}, K_{e_j} \leftarrow \text{LoadSparse}(K_{nnz}, K_e, j)$
 $S_{ji}^T \leftarrow \text{SparseGEMM}((K_{nnz_j}, K_{e_j}), Q_i^T)$
 $P_{ji}^T, O_i^T, L_i, M_i \leftarrow \text{OnlineSoftmax}(S_{ji}^T, O_i^T, L_i, M_i)$
 $P_{ji}^T \leftarrow \text{RelayoutFragment}(P_{ji}^T)$
 $V_{nnz_j}, V_{e_j} \leftarrow \text{LoadSparse}(V_{nnz}, V_e, j)$
 $O_i^T \leftarrow O_i^T + \text{SparseGEMM}((V_{nnz_j}, V_{e_j}), P_{ji}^T)$
 end for
 $O_i^T \leftarrow \text{Scale}(O_i^T, L_i)$
 Store O_i^T to HBM as O_i
end for
return O

2) *Mixed Semi-Structured Sparse Computation Workflow:*

With the support of block index map and memory pool design, in each iteration of key and value blocks, we first check the block index map to determine whether the current block is dense or sparse. If the block is dense, we load it from the dense memory pool and perform a dense GEMM operation. Otherwise, we load the nonzero elements and metadata from their respective memory pools and perform a sparse GEMM operation. The rest of the attention computation remains unchanged.

D. Efficiency Analysis

We provide a theoretical analysis of the memory compression ratio and speedup for prefill and decode phases when using *HieraSparse* with float16 2:4 sparse tensor core. Under this hardware setting, the metadata is 1/16 of the original dense tensor size. We ignore the batch and head dimension for simplicity, and denote the sequence length as L , hidden dimension as D , block size B , key block sparsity as $S_K \in [0, 1]$, and value block sparsity as $S_V \in [0, 1]$.

1) *Compression Ratio:* The overall compression rate of *HieraSparse* is defined as the ratio between the original dense KV Cache size and the compressed representation:

$$r_{comp} = \frac{Size_{baseline}}{Size_{idx} + Size_{den} + Size_{nnz} + Size_e} \quad (3)$$

block indices dense blocks non-zero blocks metadata

The total size of the original dense KV Cache is:

$$Size_{baseline} = 2 \cdot L \cdot D \quad (4)$$

When applying *HieraSparse* together with block sparsity S_K, S_V , the size of each component is:

$$Size_{idx} = 2 \cdot \frac{L}{B} \quad (5a)$$

$$Size_{den} = L \cdot D \cdot (1 - S_K) + L \cdot D \cdot (1 - S_V) \quad (5b)$$

$$Size_{nnz} = \frac{1}{2} \cdot L \cdot D \cdot S_K + \frac{1}{2} \cdot L \cdot D \cdot S_V \quad (5c)$$

$$Size_e = \frac{1}{16} \cdot L \cdot D \cdot S_K + \frac{1}{16} \cdot L \cdot D \cdot S_V \quad (5d)$$

Substituting Equations. (4) and (5) into Equation (3) yields:

$$r_{comp} = \frac{1}{1 - 0.21875 \cdot (S_K + S_V) + \frac{1}{B \cdot D}} \quad (6)$$

$$\approx \frac{1}{1 - 0.21875 \cdot (S_K + S_V)}$$

Given that block size B and hidden dimension D are usually large enough (e.g., $B = 64, D = 128$), the term $\frac{1}{B \cdot D}$ is negligible in practice. Using the approximation, $S_K = 0.5, S_V = 1.0$ yields $r_{comp} \approx 1.49\times$, while $S_K = S_V = 1.0$ gives $r_{comp} \approx 1.78\times$.

2) *Prefill Speedup:* We define the ideal speedup for prefill phase as:

$$Speedup_{prefill} = \frac{T_{baseline}}{T_{dense} + T_{sparse}} \quad (7)$$

dense blocks sparse blocks

During the prefill phase, attention is typically computation-bound because of its quadratic complexity. The two GEMM operations each cost $2L^2D$ FLOPs, totaling $4L^2D$ FLOPs (softmax and scaling are comparatively small). Let C denote the dense tensor-core throughput in FLOPs/s. The baseline time is:

$$T_{baseline} = \frac{4L^2D}{C} \quad (8)$$

With dense tensor-core throughput C FLOPs/s and sparse throughput $2C$ FLOPs/s, the time decomposes into dense and sparse parts:

$$T_{dense} = \frac{2L^2D \cdot (1 - S_K)}{C} + \frac{2L^2D \cdot (1 - S_V)}{C} \quad (9a)$$

$$T_{sparse} = \frac{2L^2D \cdot S_K}{2C} + \frac{2L^2D \cdot S_V}{2C} \quad (9b)$$

Substituting Equations (8) and (9) into Equation (7) yields:

$$Speedup_{prefill} = \frac{4}{4 - (S_K + S_V)} \quad (10)$$

3) *Decode Speedup*: During the Decode phase, the attention computation is often memory-bound due to frequent KV Cache accesses. Given the compression ratio r_{comp} derived above, the ideal speedup can be calculated as:

$$Speedup_{decode} = r_{comp} = \frac{1}{1 - 0.21875 \cdot (S_K + S_V)} \quad (11)$$

Together, these formulas quantify the ideal gains from compressing key and value cache. For example, $S_K = 0.5$ and $S_V = 1.0$ give $Speedup_{prefill} = 1.6\times$ and $Speedup_{decode} \approx 1.49\times$, while $S_K = S_V = 1.0$ yields $2.0\times$ prefill speedup and $1.78\times$ decode speedup.

IV. IMPLEMENTATION

We implement *HieraSparse* kernels with *TileLang* [48], a domain-specific language for efficient acceleration kernel development and prototyping. We integrate our kernels and memory management scheme with the popular deep learning frameworks *PyTorch* [49].

A. Sparse Tensor Core Integration

We modify the CUDA backend of *TileLang* to support sparse tensor core. We first support `float16` sparse tensor core computation with `mma.sp.m16n8k32`, and merge two dense `mma.m16n8k16` as a logical equivalent atom. This makes sure that: i) Operands can be loaded from shared memory to register using `ldmatrix.x4`. ii) Both atoms share the same fragment layout for P^T matrix as we need to dynamically switch between `mma` and `mma.sp`. Every 8 groups of 2-bit metadata data are padded to `int16` data type, which can be further vectorized into `int16x8` to fully utilize the 128-bit vectorized memory instruction.

B. Compression Kernel Implementation

We provide a suite of CUDA kernels for hierarchical compression. We use an `int16` index map, which can support up to $4M$ tokens with a block size of 64 tokens, satisfying most practical use cases. The data type can be lifted to `int32` to support $256B$ tokens, far exceeding the context length of any current LLM. Firstly, the compression kernel iterates over M sequentially along the sequence dimension and maintains block counters: for the dense part, the kernel simply applies a vectorized memory copy from the original KV Cache to the dense pool; for the sparse part, we assume that element-level mask m has been applied to the original KV Cache. Each thread first loads the original elements and selects the non-zero elements into registers, then generates the 2-bit metadata by checking the position of non-zero elements in each group. Finally, the non-zero elements and metadata are stored in the sparse pool.

Specifically for magnitude-based compression, we provide a compression kernel that fuses the mask generation and compression process. Instead of non-zero elements when compressing the sparse part, we directly select the top-2 magnitude elements in each 2:4 group, which further reduces the pruning and compression overhead.

C. Computation Kernel Implementation

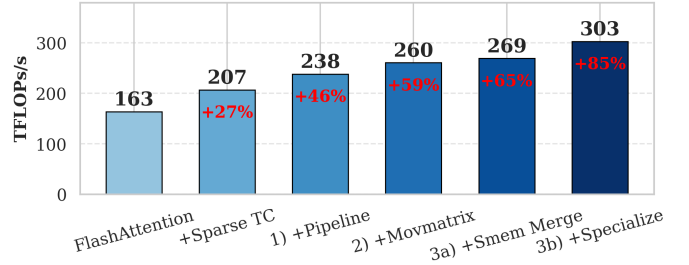


Fig. 4: The performance gain of different optimizations for prefill kernel, measured with $32K$ context and batch size of 8 with *Llama-3.1-8B-Instruct* attention setting.

Besides the sparse tensor core integration, we also implement several optimization techniques to further boost the performance of the attention kernel. For the memory-bound decode kernel, the KV Cache compression is sufficient to achieve the expected throughput. Additionally, we have a split-KV design where each thread block only processes a subset of the key and value blocks to increase parallelism. Each block outputs a partial output together with its own log-sum-exp, which are later combined in a lightweight post-processing kernel. The kernel was also optimized for Grouped-Query Attention (GQA) [50], [51], where multiple queries attending to the same KV Cache head are viewed as a short duplicated sequence and reduce the padding overhead. For the prefill phase, three main additional optimizations are implemented to fully exploit the sparse tensor core. The ablated performance gain of these techniques is shown in Figure 4, and details are as follows:

1) *Asynchronous Pipelining*: We manually control the pipelining with a double buffering between key/value loading and two GEMMs. Specifically, we utilize the `cp.async` instruction to asynchronously load the key and value blocks from HBM to shared memory. While the tensor core is performing GEMM operations on the current tile resident in one shared memory buffer, the memory unit simultaneously fetches the next tile data into the other buffer. This buffering strategy effectively hides the memory access latency, ensuring that the compute units remain highly utilized throughout the attention computation.

2) *In-fragment Re-layout*: Unlike dense attention, which takes P as the first matrix operand of $P \times V$, *HieraSparse* takes P^T as the second matrix operand in $V^T \times P^T$ and $(V_{nnz}^T, V_e^T) \times P^T$. This leads to a mandatory re-layout since the GEMM1 output operand layout is not the same as the GEMM2 input operand, which usually requires extra shared memory or multiple warp shuffling operations to transpose the data among threads. We utilize the `movmatrix` instruction to avoid this overhead. `movmatrix` is a warp-level communication instruction that operates on an 8×8 matrix atom stored in registers of all threads in a warp, allowing a row-major to column-major transpose (or vice versa) without accessing

shared memory. To apply this to a larger size matrix as shown in Figure 5, we first partition the fragment into multiple 8×8 matrix atoms, then apply the instruction multiple times among the atoms at the same location in the fragment. With this method, we can achieve the re-layout with less overhead compared to shared memory or shuffle-based methods.

3) *On-chip Memory Allocation and Specialized Kernel*: Similar to *FlashAttention*, we store the tiled query matrix in shared memory to maximize data reuse and mitigate register pressure. For key and value tensors, we first load the required blocks from global memory to shared memory, then transfer them to registers before computation. For sparse blocks, we also load the corresponding metadata to shared memory and transfer it to registers together with the non-zero elements. In a single iteration, either the shared memory of dense blocks or sparse blocks is active, so we manually overlap the allocation of them by setting the shared memory buffer pointer to the same address, which saves shared memory budget for sparse blocks.

Furthermore, we provide specialized kernels for cases where the key/value cache is fully dense or fully sparse (e.g., initial phase of generation or specific layers). In these specialized kernels, we completely remove the shared memory allocation and loading logic for the unused format (e.g., no sparse buffer allocation for fully dense kernels), allowing for larger tile sizes and better occupancy.

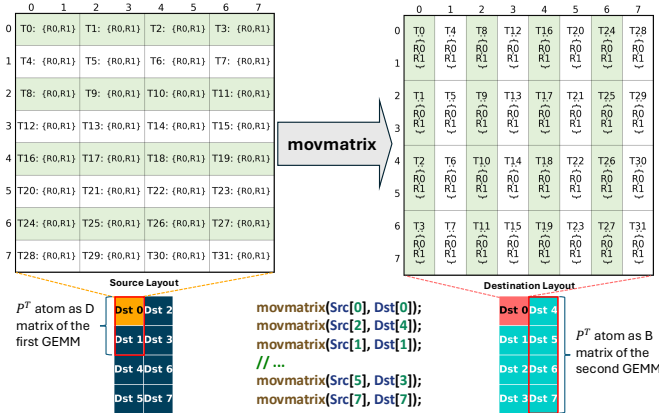


Fig. 5: The illustration of P^T fragment re-layout. The source layout consists of multiple 16×8 D-matrix atoms, and the destination layout consists of multiple 32×8 B-matrix atoms, both in row-major. They are both partitioned into 8×8 atoms, and multiple *movmatrix* are issued to perform the re-layout without shared memory access.

V. EVALUATION

We evaluate *HieraSparse* to demonstrate its effectiveness in balancing generation quality and system efficiency. Our evaluation is divided into two main parts: **Quality-Sparsity Evaluation**, which explores how different pruning strategies and sparsity settings affect the quality across various LLMs and generation stages; and **Sparsity-Efficiency Evaluation**, which assesses (i) the computational speedup in attention

kernels, (ii) the memory compression efficiency, and (iii) the end-to-end performance gains. We compare our method with *MUSTAFAR*, the state-of-the-art fine-grained KV Cache pruning approach. *MUSTAFAR* is most closely related to our work in that it performs fine-grained element-wise KV pruning. However, it is limited to the decode phase and employs unstructured sparsity with a *load-as-sparse, compute-as-dense* scheme, which prevents it from fully translating sparsity into efficiency.

All experiments are conducted on NVIDIA L40S with 48GiB DRAM. Python 3.10.19, PyTorch 2.10.0, and CUDA 12.8 are used as the software environment. The reproduction scripts for both evaluations can be found in our open-source repository.

A. Quality-Sparsity Evaluation

We first evaluated the generation quality of *HieraSparse* under different sparsity settings to find the best trade-off. To understand the impact of pruning on different inference phases, we organized our evaluation into three experimental setups: i) Applying pruning exclusively to the decode stage under various sparsity settings. ii) Applying pruning to both the prefill and decode stages using uniform sparsity settings. iii) Applying pruning to both stages with different sparsity settings, where sparsity configurations are independently chosen for prefill and decode based on the insights from the previous two setups. The generation quality was measured using *LongBench* [52], a popular comprehensive benchmark for long-context LLM evaluation, which consists of 16 tasks across 6 categories, including single-document QA, multi-document QA, summarization, few-shot learning, synthetic tasks, and code completion. We tested our method on three popular models to make sure the conclusion is general: *Llama-3.1-8B-Instruct*, *Mistral-7B-Instruct-v0.2*, and *Qwen3-8B*. We didn't include *MUSTAFAR* results for *Qwen3-8B* since the model was not supported by the official implementation. For space reasons, we report the results of 6 categories and an averaged score; complete results of 16 tasks can be found in our repository. Following the common settings in prior work [17], [18], [23], [24], we kept the first 64 “sink” tokens and the last 256 “local window” tokens dense, and pruned the remaining tokens with different sparsity.

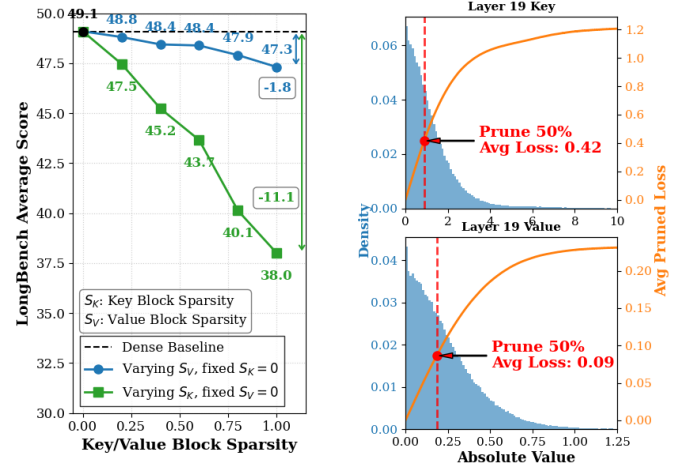
1) *Pruning on Decode Stage Only*: *MUSTAFAR* has performed a comprehensive evaluation on KV Cache during the decode phase, showing that the generation quality can be well preserved even pruned to 50% sparsity. Hence, we compared the *LongBench* scores between dense, *MUSTAFAR*, *HieraSparse* under different sparsity settings. To ensure a fair comparison across methods, we standardize hyperparameters (S_K and S_V for *HieraSparse*, and K and V for *MUSTAFAR*) by selecting configurations that achieve the same sparsity level, defined as the proportion of zero entries in the key and value caches. As shown in Table III, *HieraSparse* is able to maintain generation quality compared to the dense baselines across different models. For instance, under 50% Value sparsity, the average score drops by only 0.06 on *Llama-3.1-8B-Instruct*

TABLE III: *LongBench* score under different methods and sparsity settings when applying to decode stage.

Method	Hyperparam	Sparsity		Comp. Rate		LongBench Score							Attn. Spd.
		Key	Value	Key	Value	Single-Doc	Multi-Doc	Summ.	Few-Shot	Synth.	Code	Avg.	
Llama-3.1-8B-Instruct													
Dense	—	0%	0%	1.0×	1.0×	42.87	44.65	29.21	69.31	53.71	60.03	49.96	1.00×
MUSTAFAR	$K0.0, V0.5$	0%	50%	1.0×	1.5×	42.89	44.62	28.95	69.51	53.73	59.95	49.94	0.32×
HieraSparse	$S_K0.0, S_V1.0$	0%	50%	1.0×	1.8×	43.60	44.69	28.47	69.23	53.91	59.67	49.90	1.28×
MUSTAFAR	$K0.5, V0.0$	50%	0%	1.5×	1.0×	43.06	44.26	28.83	69.19	53.94	59.97	49.88	0.32×
HieraSparse	$S_K1.0, S_V0.0$	50%	0%	1.8×	1.0×	40.90	43.23	26.13	68.16	52.19	57.07	47.95	1.27×
MUSTAFAR	$K0.5, V0.5$	50%	50%	1.5×	1.5×	42.95	44.02	28.35	69.17	53.33	59.69	49.58	0.37×
HieraSparse	$S_K1.0, S_V1.0$	50%	50%	1.8×	1.8×	40.96	43.21	26.03	67.69	52.83	56.57	47.88	1.71×
Mistral-7B-Instruct-v0.2													
Dense	—	0%	0%	1.0×	1.0×	32.27	25.78	27.91	66.72	46.45	54.96	42.34	1.00×
MUSTAFAR	$K0.0, V0.5$	0%	50%	1.0×	1.5×	35.88	30.18	27.58	66.73	42.31	54.80	42.91	0.32×
HieraSparse	$S_K0.0, S_V1.0$	0%	50%	1.0×	1.8×	32.22	25.64	27.31	66.87	43.99	54.71	41.79	1.28×
MUSTAFAR	$K0.5, V0.0$	50%	0%	1.5×	1.0×	36.32	30.23	27.96	66.70	43.52	54.91	43.27	0.32×
HieraSparse	$S_K1.0, S_V0.0$	50%	0%	1.8×	1.0×	29.78	23.61	25.06	66.15	37.66	53.24	39.25	1.27×
MUSTAFAR	$K0.5, V0.5$	50%	50%	1.5×	1.5×	36.28	30.40	27.84	66.65	41.92	54.79	42.98	0.37×
HieraSparse	$S_K1.0, S_V1.0$	50%	50%	1.8×	1.8×	29.41	21.96	24.37	65.36	33.92	53.19	38.03	1.71×
Qwen3-8B													
Dense	—	0%	0%	1.0×	1.0×	42.49	46.30	27.54	68.78	50.75	65.83	50.28	1.00×
HieraSparse	$S_K0.0, S_V1.0$	0%	50%	1.0×	1.8×	42.04	46.77	26.29	68.08	50.50	66.10	49.96	1.28×
HieraSparse	$S_K1.0, S_V0.0$	50%	0%	1.8×	1.0×	39.09	43.68	23.04	65.37	50.75	64.93	47.81	1.27×
HieraSparse	$S_K1.0, S_V1.0$	50%	50%	1.8×	1.8×	37.26	42.92	22.59	64.54	50.75	64.11	47.03	1.71×

and 0.32 on *Qwen3-8B*. Even when configuring both key and value caches to 50% sparsity, the models retain strong performance with manageable degradation (e.g., a 2.08 drop on *Llama-3.1-8B-Instruct*). As expected, the semi-structured nature of our method introduces a slight quality degradation compared to the unstructured *MUSTAFAR* (e.g., trailing by 0.04 to 1.70 on *Llama-3.1-8B-Instruct* depending on the sparsity configuration). However, these quality trade-offs are well justified by a significant efficiency gain of up to 4.57× against *MUSTAFAR*. We also surprisingly observed that *MUSTAFAR* exhibits lower performance than the dense baseline, and the detailed efficiency evaluation will be presented in Section V-B. Additionally, we observed that *Mistral-7B-Instruct-v0.2* is more sensitive to our structural pruning than the other two models, showing a larger drop of up to 4.31 against the dense baseline when directly applying the same setting, which could be mitigated by tuning the sparsity settings.

2) *Uniformed Sparsity Pruning on Both Prefill and Decode*: As shown in Figure 6a, we measured the overall *LongBench* scores in two settings: i) Keep all value cache as dense, gradually increase key block sparsity S_K (green line). ii) Keep all key cache as dense, gradually increase value block sparsity S_V (blue line). The results indicate that pruning the key cache is much more sensitive than pruning the value cache, given the same block sparsity. By keeping the key cache dense and pruning the value cache to sparse, the overall score of *LongBench* only drops around 1.8, while keeping value cache dense and pruning key cache results in a much larger drop of 11.1. By inspecting the numeric distribution of key and value in Figure 6b, we conclude that this is mainly due to: i) Key and value exhibit different magnitude distributions. Key cache



(a) The average *LongBench* scores with different sparsity settings when extending pruned cache to prefill stage.

(b) The distribution of the key and value magnitude and respective average loss when pruning to semi-structured.

Fig. 6: The quality evaluation of *HieraSparse* when extended to prefill stage.

consistently shows much greater magnitude than value cache, which can lead to up to 4.8× magnitude loss when pruning lower 50% elements. ii) The calculation of the attention score involves exponential operations in softmax, which amplifies the impact of errors in key states; in contrast, errors in value representations have a more direct and linear effect on the final output. Thus, for magnitude-based pruning during the prefill stage, we keep the key cache dense to maintain generation

quality.

TABLE IV: Average *LongBench* score and attention speedup under different sparsity settings for prefill and decode.

Hyperparam		Attn. Spd.		Avg. Score
Prefill	Decode	Prefill	Decode	
Llama-3.1-8B-Instruct				
$S_K 0.0, S_V 1.0$	$S_K 0.0, S_V 1.0$	1.34×	1.28×	47.32
$S_K 0.0, S_V 1.0$	$S_K 1.0, S_V 1.0$	1.34×	1.71×	45.59
Mistral-7B-Instruct-v0.2				
$S_K 0.0, S_V 1.0$	$S_K 0.0, S_V 1.0$	1.34×	1.28×	41.30
$S_K 0.0, S_V 1.0$	$S_K 1.0, S_V 1.0$	1.34×	1.71×	38.18
Qwen3-8B				
$S_K 0.0, S_V 1.0$	$S_K 0.0, S_V 1.0$	1.34×	1.28×	48.01
$S_K 0.0, S_V 1.0$	$S_K 1.0, S_V 1.0$	1.34×	1.71×	45.10

3) *Differentiated Sparsity Pruning on Both Prefill and Decode*: We fix the prefill sparsity to $S_K 0.0, S_V 1.0$ and test different decode sparsity by either keeping the sparsity same or further pruning all key cache to $S_K 1.0, S_V 1.0$. The results show that aggressively pruning the decode key cache further boosts decode speedup from 1.28×

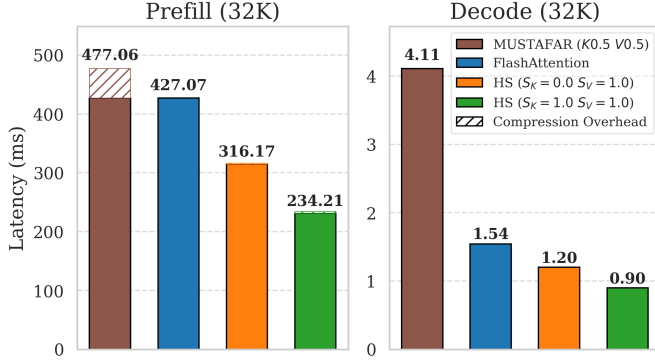


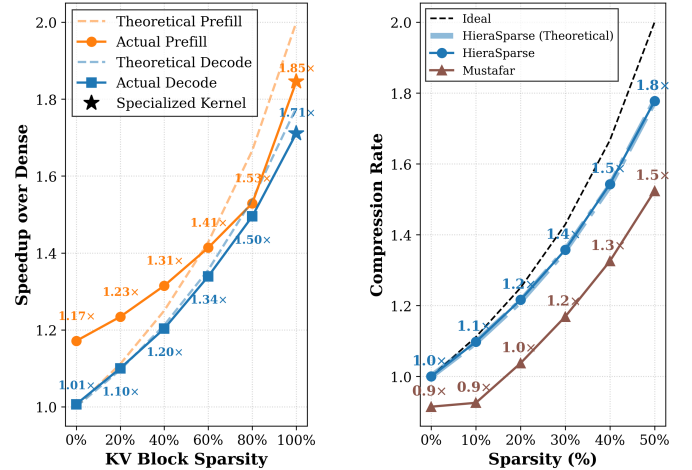
Fig. 7: Comparison of attention kernel latency, including compression overhead. The overhead of our method (HS) is minimal and barely visible in the figure.

B. Sparsity-Efficiency Evaluation

In this section, we perform a detailed efficiency analysis without considering the exact pruning algorithm to explore the potential performance gain from *HieraSparse*. We benchmark the performance from different perspectives: i) Kernel performance. ii) Memory compression efficiency. iii) Per-layer and end-to-end performance when considering other computations during inference.

1) *Kernel Performance*: We first benchmark the kernel under prefill and decode stages of *Llama-3.1-8B-Instruct* with a batch size of 8 and context length of 32K, and compare the execution latency, as shown in Figure 7 (the same kernel

acceleration results are also reported in Table III and Table IV). We also included the compression overhead of both systems, which contributes to prefill latency. The results indicate that when applying to the prefill stage, *HieraSparse* achieved a maximum 1.85×



(a) Kernel speedup against dense baseline for different block sparsity.

(b) Comparison between *HieraSparse* and *MUSTAFAR* on compression rate.

Fig. 8: The efficiency evaluation of *HieraSparse* under different sparsity.

Cross-GPU portability. Although our main study uses the L40S, the kernels depend only on the `mma.sp` and `movmatrix` instructions, which have been available since the Ampere (SM80) and Turing (SM75) architectures, respectively, and are forward-compatible with all later generations.

To verify that the speedups generalize beyond a single device, we evaluate our kernels on three additional GPUs spanning the Ampere and Ada generations and observe consistent attention-kernel speedups over the dense FP16 baseline: A100 ($1.45\times$ prefill, $1.74\times$ decode), RTX 4090 ($1.80\times / 1.72\times$), and A40 ($1.79\times / 1.81\times$). Since our roofline analysis depends only on the presence of sparse tensor cores, the speedups transfer across hardware once parameters are tuned to each device’s SM count and on-chip SRAM.

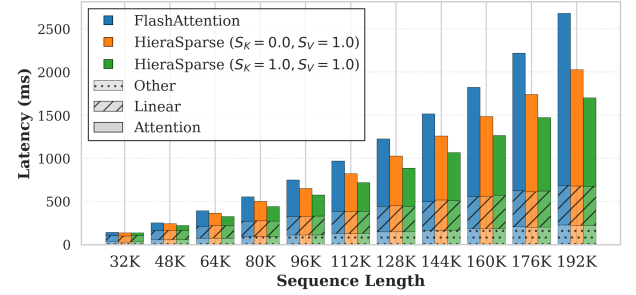
We also benchmark kernel speedup across block sparsity levels, as shown in Figure 8a. The decode kernel speedup closely follows the theoretical curve, with a small gap because non-memory operation latencies are excluded from the model. In contrast, the prefill speedup is slightly offset: at low sparsity it exceeds the theoretical estimate, likely due to reduced memory traffic and higher L2 cache hit rate. The trend is also flatter than theory because, even after overlapping sparse and dense shared memory usage, the total shared memory is still at least the same as a dense counterpart kernel, which limits occupancy and thus speedup; this discrepancy disappears at 100% block sparsity, where a specialized kernel with much lower shared memory consumption can be used.

2) *Memory Compression Efficiency*: Besides computation speedup, another important benefit of pruning the KV Cache is the reduction of memory usage. As shown in Figure 8b, we measured the compression rate of *HieraSparse* under different sparsity settings, and compared the actual compression rate with the ideal compression rate, the theoretical compression rate of *HieraSparse*, and *MUSTAFAR* compression rate. The results show that our method can achieve exactly its theoretical compression rate, and up to $1.2\times$ compression rate compared with *MUSTAFAR* under the same sparsity.

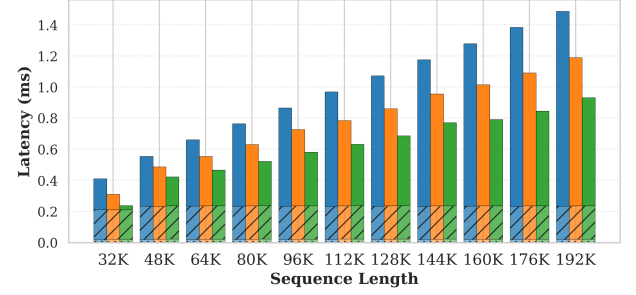
3) *Layer-wise and End-to-End Performance*: We also measured the end-to-end performance of *HieraSparse* with different sequence lengths, and compared the latency with the dense baseline on *Llama-3.1-8B-Instruct*. First, we present the per-layer latency breakdown in Figure 9, where the latency of a layer is decoupled into Attention, Linear, and Other. The results show different components during the inference and the attention acceleration of *HieraSparse* under different sequence lengths. Furthermore, we report end-to-end results including computation latencies and memory consumption from 32K to 160K in Table V, using chunked prefill (chunk size is set to 32K, as in *DuoAttention*) to support long sequences; the results beyond 160K are omitted due to dense inference being out of memory. The results of end-to-end are in a similar trend as per-layer breakdown, as it’s consist of multiple layer inference.

VI. CONCLUSION AND FUTURE WORK

In this paper, we presented *HieraSparse*, a hierarchical KV cache compression framework that accelerates memory usage and attention computation in LLMs. By utilizing GPU sparse tensor cores, *HieraSparse* efficiently converts sparsity into computational speedups for semi-structured attention during both the prefill and decode stages. Our experiments showed



(a) Per-layer prefill latency breakdown against sequence length.



(b) Per-layer decode latency breakdown against sequence length.

Fig. 9: Per-layer latency breakdown for prefill and decode.

that *HieraSparse* achieves substantial attention speedups and higher KV compression ratios compared to state-of-the-art unstructured sparsity methods, while preserving a flexible balance between generation quality and sparsity.

Looking ahead, we see several promising directions for future work. First, our current magnitude-based pruning approach leaves some acceleration potential of the prefill kernels untapped. Exploring more sophisticated offline pruning methods could be especially useful in scenarios with prefix caching. Second, adapting our kernels to support fine-grained, unstructured sparsity is an exciting opportunity. Recent work, such as *TASDER* [53] and *VENOM* [54], shows that unstructured sparsity can be efficiently mapped to structured sparsity accelerators. This suggests we could add dynamic fine-grained patterns without losing hardware efficiency. Third, combining *HieraSparse* with quantization or coarse-grained pruning could further improve LLM inference, particularly on devices with limited resources. Finally, while our kernels already run across the Ampere and Ada generations, extending them to newer architectures such as Hopper and Blackwell, whose sparse tensor cores expose `wgmma.sp` and `tcgen05.mma.sp`, can unlock further gains; reaching parity with heavily optimized dense kernels on these platforms requires additional engineering such as TMA-based pipelining and warp specialization, though the memory savings are guaranteed regardless of the target hardware.

VII. ACKNOWLEDGMENTS

This research is supported by the Ministry of Education, Singapore, under its Academic Research Fund Tier 1

TABLE V: End-to-end performance comparison on *Llama-3.1-8B-Instruct*.

Context	Prefill Method	TTFT (Speedup)	Decode Method	TPOT (Speedup)	Key Mem	Value Mem	Peak Mem
32k	Dense	4.7s	Dense	40.0ms	2.00GiB	2.00GiB	22.62GiB
	$S_K 0.0, S_V 1.0$	4.1s (1.14 $\times$)	$S_K 0.0, S_V 1.0$	35.8ms (1.12 $\times$)	2.00GiB	1.12GiB	21.74GiB
	$S_K 1.0, S_V 1.0$	3.8s (1.23 $\times$)	$S_K 1.0, S_V 1.0$	30.7ms (1.30 $\times$)	1.12GiB	1.12GiB	20.87GiB
64k	Dense	13.3s	Dense	60.3ms	4.00GiB	4.00GiB	26.62GiB
	$S_K 0.0, S_V 1.0$	11.2s (1.18 $\times$)	$S_K 0.0, S_V 1.0$	50.8ms (1.19 $\times$)	4.00GiB	2.25GiB	24.87GiB
	$S_K 1.0, S_V 1.0$	10.2s (1.30 $\times$)	$S_K 1.0, S_V 1.0$	42.6ms (1.41 $\times$)	2.25GiB	2.25GiB	23.12GiB
96k	Dense	25.3s	Dense	79.5ms	6.00GiB	6.00GiB	30.62GiB
	$S_K 0.0, S_V 1.0$	20.7s (1.22 $\times$)	$S_K 0.0, S_V 1.0$	65.8ms (1.21 $\times$)	6.00GiB	3.38GiB	28.00GiB
	$S_K 1.0, S_V 1.0$	18.6s (1.35 $\times$)	$S_K 1.0, S_V 1.0$	53.3ms (1.49 $\times$)	3.38GiB	3.38GiB	25.37GiB
128k	Dense	39.8s	Dense	98.4ms	8.00GiB	8.00GiB	34.62GiB
	$S_K 0.0, S_V 1.0$	32.5s (1.22 $\times$)	$S_K 0.0, S_V 1.0$	80.7ms (1.22 $\times$)	8.00GiB	4.50GiB	31.12GiB
	$S_K 1.0, S_V 1.0$	28.8s (1.41 $\times$)	$S_K 1.0, S_V 1.0$	64.0ms (1.54 $\times$)	4.50GiB	4.50GiB	27.62GiB
160k	Dense	59.4s	Dense	117.3 ms	10.00GiB	10.00GiB	38.63GiB
	$S_K 0.0, S_V 1.0$	47.1s (1.26 $\times$)	$S_K 0.0, S_V 1.0$	95.6ms (1.23 $\times$)	10.00GiB	5.62GiB	34.25GiB
	$S_K 1.0, S_V 1.0$	40.9s (1.45 $\times$)	$S_K 1.0, S_V 1.0$	74.8ms (1.57 $\times$)	5.62GiB	5.62GiB	29.87GiB

(#026632-00001, RS57/25), and by the NTU Singapore start up grant (#025593-00001).

We acknowledge the use of *Anthropic Claude* and *Google Gemini* for linguistic polishing and grammatical corrections. All core technical contributions, structural integrity, and references remain the original work of the human authors.

REFERENCES

- [1] OpenAI, “GPT-4 Technical Report,” 2024. [Online]. Available: <https://doi.org/10.48550/arXiv.2303.08774>
- [2] MetaAI, “The Llama 3 Herd of Models,” 2024. [Online]. Available: <https://doi.org/10.48550/arXiv.2407.21783>
- [3] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv, C. Zheng, D. Liu, F. Zhou, F. Huang, F. Hu, H. Ge, H. Wei, H. Lin, J. Tang, J. Yang, J. Tu, J. Zhang, J. Yang, J. Yang, J. Zhou, J. Zhou, J. Lin, K. Dang, K. Bao, K. Yang, L. Yu, L. Deng, M. Li, M. Xue, M. Li, P. Zhang, P. Wang, Q. Zhu, R. Men, R. Gao, S. Liu, S. Luo, T. Li, T. Tang, W. Yin, X. Ren, X. Wang, X. Zhang, X. Ren, Y. Fan, Y. Su, Y. Zhang, Y. Zhang, Y. Wan, Y. Liu, Z. Wang, Z. Cui, Z. Zhang, Z. Zhou, and Z. Qiu, “Qwen3 Technical Report,” 2025. [Online]. Available: <https://doi.org/10.48550/arXiv.2505.09388>
- [4] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. de las Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier, L. R. Lavaud, M.-A. Lachaux, P. Stock, T. L. Scao, T. Lavril, T. Wang, T. Lacroix, and W. E. Sayed, “Mistral 7b,” 2023. [Online]. Available: <https://doi.org/10.48550/arXiv.2310.06825>
- [5] J. Jiang, F. Wang, J. Shen, S. Kim, and S. Kim, “A survey on large language models for code generation,” 2024. [Online]. Available: <https://doi.org/10.48550/arXiv.2406.00515>
- [6] T. Zhang, F. Ladhak, E. Durmus, P. Liang, K. McKeown, and T. B. Hashimoto, “Benchmarking Large Language Models for News Summarization,” 2023. [Online]. Available: <https://doi.org/10.48550/arXiv.2301.13848>
- [7] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, “Chain-of-thought prompting elicits reasoning in large language models,” 2023. [Online]. Available: <https://doi.org/10.48550/arXiv.2201.11903>
- [8] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. tau Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive nlp tasks,” 2021. [Online]. Available: <https://doi.org/10.48550/arXiv.2005.11401>
- [9] R. Agarwal, A. Singh, L. M. Zhang, B. Bohnet, L. Rosias, S. Chan, B. Zhang, A. Anand, Z. Abbas *et al.*, “Many-shot in-context learning,” 2024. [Online]. Available: <https://doi.org/10.48550/arXiv.2404.11018>
- [10] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez, “Memgpt: Towards llms as operating systems,” 2023. [Online]. Available: <https://doi.org/10.48550/arXiv.2310.08560>
- [11] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu, “Roformer: Enhanced transformer with rotary position embedding,” 2023. [Online]. Available: <https://doi.org/10.48550/arXiv.2104.09864>
- [12] W. Xiong, J. Liu, I. Molybog, H. Zhang, P. Bhargava, R. Hou, L. Martin, R. Rungta, K. A. Sankararaman, B. Oguz, M. Khabisa, H. Fang, Y. Mehdad, S. Narang, K. Malik, A. Fan, S. Bhosale, S. Edunov, M. Lewis, S. Wang, and H. Ma, “Effective long-context scaling of foundation models,” 2023. [Online]. Available: <https://doi.org/10.48550/arXiv.2309.16039>
- [13] L. Pekelis, M. Feil, F. Moret, M. Huang, and T. Peng, “Llama 3 gradient: A series of long context model,” [Online]. Available: <https://huggingface.co/gradientai/Llama-3-8B-Instruct-Gradient-1048k>
- [14] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” 2023. [Online]. Available: <https://doi.org/10.48550/arXiv.1706.03762>
- [15] Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, R. Cai, Z. Song, Y. Tian, C. Ré, C. Barrett, Z. Wang, and B. Chen, “H₂o: Heavy-hitter oracle for efficient generative inference of large language models,” 2023. [Online]. Available: <https://doi.org/10.48550/arXiv.2306.14048>
- [16] Y. Li, Y. Huang, B. Yang, B. Venkitesh, A. Locatelli, H. Ye, T. Cai, P. Lewis, and D. Chen, “Snapkv: Llm knows what you are looking for before generation,” 2024. [Online]. Available: <https://doi.org/10.48550/arXiv.2404.14469>
- [17] Y. Xu, Z. Jie, H. Dong, L. Wang, X. Lu, A. Zhou, A. Saha, C. Xiong, and D. Sahoo, “Think: Thinner key cache by query-driven pruning,” 2025. [Online]. Available: <https://doi.org/10.48550/arXiv.2407.21018>
- [18] Y. Zhang, Z. He, H. Jiang, C. Zhang, Y. Yang, J. Wang, and L. Qiu, “Leank: Learnable k cache channel pruning for efficient decoding,” 2025. [Online]. Available: <https://doi.org/10.48550/arXiv.2508.02215>
- [19] Y. Fu, Z. Cai, A. Asi, W. Xiong, Y. Dong, and W. Xiao, “Not all heads matter: A head-level kv cache compression method with integrated retrieval and reasoning,” 2025. [Online]. Available: <https://doi.org/10.48550/arXiv.2410.19258>
- [20] Y. Feng, J. Lv, Y. Cao, X. Xie, and S. K. Zhou, “Ada-kv: Optimizing kv cache eviction by adaptive budget allocation for efficient llm inference,” 2025. [Online]. Available: <https://doi.org/10.48550/arXiv.2407.11550>
- [21] Z. Cai, Y. Zhang, B. Gao, Y. Liu, Y. Li, T. Liu, K. Lu, W. Xiong, Y. Dong, J. Hu, and W. Xiao, “Pyramidkv: Dynamic kv cache compression based on pyramidal information funneling,” 2025. [Online]. Available: <https://doi.org/10.48550/arXiv.2406.02069>
- [22] X. Zhou, W. Wang, M. Zeng, J. Guo, X. Liu, L. Shen, M. Zhang, and L. Ding, “Dynamickv: Task-aware adaptive kv cache compression for long context llms,” 2025. [Online]. Available: <https://doi.org/10.48550/arXiv.2412.14838>
- [23] G. Xiao, J. Tang, J. Zuo, J. Guo, S. Yang, H. Tang, Y. Fu, and S. Han, “Duoattention: Efficient long-context llm inference with retrieval and streaming heads,” 2024. [Online]. Available: <https://doi.org/10.48550/arXiv.2410.10819>
- [24] D. Joo, H. Hosseini, R. Hadidi, and B. Asgari, “Mustafar: Promoting

- unstructured sparsity for kv cache pruning in llm inference,” 2025. [Online]. Available: <https://doi.org/10.48550/arXiv.2505.22913>
- [25] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, “Efficient streaming language models with attention sinks,” 2024. [Online]. Available: <https://doi.org/10.48550/arXiv.2309.17453>
- [26] P. Singhanian, S. Singh, S. He, S. Feizi, and A. Bhatele, “Loki: Low-rank keys for efficient sparse attention,” 2024. [Online]. Available: <https://doi.org/10.48550/arXiv.2406.02542>
- [27] S. Yang, Y. Sheng, J. E. Gonzalez, I. Stoica, and L. Zheng, “Post-training sparse attention with double sparsity,” 2024. [Online]. Available: <https://doi.org/10.48550/arXiv.2408.07092>
- [28] H. Wang, Z. Zhang, and S. Han, “Spatten: Efficient sparse attention architecture with cascade token and head pruning,” in *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2021, pp. 97–110. [Online]. Available: <https://doi.org/10.48550/arXiv.2012.09852>
- [29] N. Zheng, B. Lin, Q. Zhang, L. Ma, Y. Yang, F. Yang, Y. Wang, M. Yang, and L. Zhou, “SparTA: Deep-Learning model sparsity via Tensor-with-Sparsity-Attribute,” in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. Carlsbad, CA: USENIX Association, Jul. 2022, pp. 213–232. [Online]. Available: <https://www.usenix.org/conference/osdi22/presentation/zheng-ningxin>
- [30] D. Joo, H. Hosseini, R. Hadidi, and B. Asgari, *Coruscant: Co-Designing GPU Kernel and Sparse Tensor Core to Advocate Unstructured Sparsity in Efficient LLM Inference*. New York, NY, USA: Association for Computing Machinery, 2025, p. 232–245. [Online]. Available: <https://doi.org/10.1145/3725843.3756065>
- [31] H. Xia, Z. Zheng, Y. Li, D. Zhuang, Z. Zhou, X. Qiu, Y. Li, W. Lin, and S. L. Song, “Flash-llm: Enabling cost-effective and highly-efficient large generative model inference with unstructured sparsity,” *Proc. VLDB Endow.*, vol. 17, no. 2, p. 211–224, Oct. 2023. [Online]. Available: <https://doi.org/10.14778/3626292.3626303>
- [32] R. Fan, X. Yu, P. Dong, Z. Li, G. Gong, Q. Wang, W. Wang, and X. Chu, “Spinfer: Leveraging low-level sparsity for efficient large language model inference on gpus,” in *Proceedings of the Twentieth European Conference on Computer Systems*, ser. EuroSys ’25. New York, NY, USA: Association for Computing Machinery, 2025, p. 243–260. [Online]. Available: <https://doi.org/10.1145/3689031.3717481>
- [33] A. Mishra, J. A. Latorre, J. Pool, D. Stosic, D. Stosic, G. Venkatesh, C. Yu, and P. Micikevicius, “Accelerating sparse deep neural networks,” 2021. [Online]. Available: <https://doi.org/10.48550/arXiv.2104.08378>
- [34] Nvidia, “Ampere tensor core,” <https://www.nvidia.com/en-us/data-center/ampere-architecture/>, 2023.
- [35] Amd, “Amd matrix core,” <https://www.amd.com/content/dam/amd/en/documents/instinct-tech-docs/product-briefs/instinct-mi325x-datasheet.pdf>, 2023.
- [36] I. Beltagy, M. E. Peters, and A. Cohan, “Longformer: The long-document transformer,” 2020. [Online]. Available: <https://doi.org/10.48550/arXiv.2004.05150>
- [37] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, “Flashattention: Fast and memory-efficient exact attention with io-awareness,” 2022. [Online]. Available: <https://doi.org/10.48550/arXiv.2205.14135>
- [38] M. N. Rabe and C. Staats, “Self-attention does not need $o(n^2)$ memory,” 2022. [Online]. Available: <https://doi.org/10.48550/arXiv.2112.05682>
- [39] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with pagedattention,” in *Proceedings of the 29th Symposium on Operating Systems Principles*, ser. SOSP ’23. New York, NY, USA: Association for Computing Machinery, 2023, p. 611–626. [Online]. Available: <https://doi.org/10.1145/3600006.3613165>
- [40] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng, “Sglang: Efficient execution of structured language model programs,” 2024. [Online]. Available: <https://doi.org/10.48550/arXiv.2312.07104>
- [41] J. Yao, H. Li, Y. Liu, S. Ray, Y. Cheng, Q. Zhang, K. Du, S. Lu, and J. Jiang, “Cacheblend: Fast large language model serving for rag with cached knowledge fusion,” 2025. [Online]. Available: <https://doi.org/10.48550/arXiv.2405.16444>
- [42] I. Gim, G. Chen, S. seob Lee, N. Sarda, A. Khandelwal, and L. Zhong, “Prompt cache: Modular attention reuse for low-latency inference,” 2024. [Online]. Available: <https://doi.org/10.48550/arXiv.2311.04934>
- [43] J. Yang, B. Hou, W. Wei, Y. Bao, and S. Chang, “Kvlink: Accelerating large language models via efficient kv cache reuse,” 2025. [Online]. Available: <https://doi.org/10.48550/arXiv.2502.16002>
- [44] Z. Liu, J. Yuan, H. Jin, S. H. Zhong, Z. Xu, V. Braverman, B. Chen, and X. Hu, “Kivi: a tuning-free asymmetric 2bit quantization for kv cache,” in *Proceedings of the 41st International Conference on Machine Learning*, ser. ICML’24. JMLR.org, 2024. [Online]. Available: <https://doi.org/10.48550/arXiv.2402.02750>
- [45] T. Zhang, J. Yi, Z. Xu, and A. Shrivastava, “Kv cache is 1 bit per channel: Efficient large language model inference with coupled quantization,” 2024. [Online]. Available: <https://doi.org/10.48550/arXiv.2405.03917>
- [46] J.-H. Kim, J. Kim, S. Kwon, J. W. Lee, S. Yun, and H. O. Song, “Kvzip: Query-agnostic kv cache compression with context reconstruction,” 2025. [Online]. Available: <https://doi.org/10.48550/arXiv.2505.23416>
- [47] Z. Chen, Y. Quan, Z. Qu, L. Liu, Y. Ding, and Y. Xie, “Dynamic n:m fine-grained structured sparse attention mechanism,” 2022. [Online]. Available: <https://arxiv.org/abs/2203.00091>
- [48] L. Wang, Y. Cheng, Y. Shi, Z. Tang, Z. Mo, W. Xie, L. Ma, Y. Xia, J. Xue, F. Yang, and Z. Yang, “Tilang: A composable tiled programming model for ai systems,” 2025. [Online]. Available: <https://doi.org/10.48550/arXiv.2504.17577>
- [49] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, “Pytorch: An imperative style, high-performance deep learning library,” 2019. [Online]. Available: <https://doi.org/10.48550/arXiv.1912.01703>
- [50] N. Shazeer, “Fast transformer decoding: One write-head is all you need,” 2019. [Online]. Available: <https://doi.org/10.48550/arXiv.1911.02150>
- [51] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, “Gqa: Training generalized multi-query transformer models from multi-head checkpoints,” 2023. [Online]. Available: <https://doi.org/10.48550/arXiv.2305.13245>
- [52] Y. Bai, X. Lv, J. Zhang, H. Lyu, J. Tang, Z. Huang, Z. Du, X. Liu, A. Zeng, L. Hou, Y. Dong, J. Tang, and J. Li, “LongBench: A bilingual, multitask benchmark for long context understanding,” in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, L.-W. Ku, A. Martins, and V. Srikumar, Eds. Bangkok, Thailand: Association for Computational Linguistics, Aug. 2024, pp. 3119–3137. [Online]. Available: <https://doi.org/10.48550/arXiv.2308.14508>
- [53] G. Jeong, P.-A. Tsai, A. R. Bambhaniya, S. W. Keckler, and T. Krishna, “Enabling unstructured sparse acceleration on structured sparse accelerators,” 2025. [Online]. Available: <https://doi.org/10.48550/arXiv.2403.07953>
- [54] R. L. Castro, A. Ivanov, D. Andrade, T. Ben-Nun, B. B. Fraguera, and T. Hoeffer, “Venom: A vectorized n:m format for unleashing the power of sparse tensor cores,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, ser. SC ’23. New York, NY, USA: Association for Computing Machinery, 2023. [Online]. Available: <https://doi.org/10.1145/3581784.3607087>